

词典编码详解

词典编码你可以把它理解成一句话：

如果数据里有重复出现的字符串，就不要每次都原样存，而是用一个“指针”或“编号”去代替它。

比如：

吃葡萄不吐葡萄皮，不吃葡萄倒吐葡萄皮

里面“葡萄”“葡萄皮”重复出现很多次。

如果我们用一个代号表示“葡萄”：

葡萄 = X

原句就可以变短。

这就是词典编码的基本思想：

重复字符串 → 用更短的引用方式表示



1. 词典编码是什么

词典编码主要利用的是数据中的：

重复代码串

它常用于：

文本文件
光栅图像
一般二进制数据

词典编码里的“词典”不是语文词典，而是：

字符串 ↔ 代号/指针/索引

的对应关系。

词典编码要解决两个问题

第一：

词典怎么形成？

是提前给好，还是边读数据边动态建立？

第二：

重复字符串怎么表示？

是输出指向过去位置的指针，还是输出词典里的编号？

这两个问题就引出了两类词典编码。

2. 第一类词典法

第一类词典法的思想是：

在已经编码过的数据中，查找当前要编码的字符串是否出现过。
如果出现过，就用一个指针指向过去出现的位置。

也就是说，第一类词典法的词典来自：

已经出现过的数据

它不一定真的维护一张“词典表”，而是把前面已经读过的一段数据当成词典。

第一类词典法的代表算法是：

LZ77
LZSS

第一类词典法的核心

可以理解成：

前面出现过 → 不重复写 → 指回去

例如当前要压缩：

```
abcabcabc
```

当读到后面的 `abc` 时，编码器会发现：

这个 `abc` 前面出现过

于是可以输出类似：

回到前面某个位置，复制 3 个字符

这样就不用再次存储 `abc` 本身。

3. 第二类词典法

第二类词典法的思想是：

在编码过程中动态创建一个短语词典。

如果后面遇到词典里已经有的短语，就输出这个短语的索引号。

也就是说，第二类词典法的词典来自：

编码过程中逐渐建立的短语表

第二类词典法的代表算法是：

LZ78

LZW

第二类词典法的核心

可以理解成：

边读数据 → 边建立短语表 → 以后遇到短语就输出编号

例如词典里已经有：

```
1 = A
2 = B
3 = AB
4 = ABA
```

后面再遇到 ABA，就可以输出：

```
4
```

而不是输出三个字符 A B A。

4. 第一类和第二类的区别

对比点	第一类词典法	第二类词典法
代表算法	LZ77、LZSS	LZ78、LZW
词典来源	已经编码过的历史数据	动态建立的短语词典
输出形式	指向历史位置的指针	词典中短语的索引号
核心动作	在滑动窗口里找最长匹配	在词典表里查已有短语
直观理解	“前面有，回去复制”	“表里有，输出编号”

最关键的区别是：

```
第一类：指向过去出现的位置
第二类：指向动态词典里的编号
```

5. LZ77 是什么

LZ77 属于第一类词典编码。

它的核心机制是：

```
滑动窗口
```

可以把 LZ77 的编码过程想成编码器面前有两部分：

已经编码过的数据：搜索窗口
还没有编码的数据：前向缓冲区

示意：

[已经编码过的数据][即将编码的数据]
 ↑ ↑
 搜索窗口 前向缓冲区

LZ77 会在搜索窗口中查找：

前向缓冲区开头的字符串，是否在过去出现过

如果出现过，就输出一个三元组。

6. LZ77 输出格式

LZ77 常见输出格式是：

`(off, len, c)`

其中：

off：匹配串在窗口中的位置或偏移
len：匹配的长度
c：匹配串后面的第一个未匹配字符

它的意思是：

从窗口中 **off** 对应的位置开始，复制 **len** 个字符，
然后再补上字符 **c**。

如果没有找到匹配：

`(0, 0, c)`

表示：

没有可复制的内容，直接输出字符 `c`。

注意：不同教材里 `off` 的定义可能略有不同。

有的教材把 `off` 理解为“距离当前位置往回数多少位”，有的教材把它理解为“相对窗口左边界的偏移”。考试时按老师课件里的定义写即可。

7. LZ77 怎么计算

LZ77 编码步骤如下：

1. 从当前编码位置开始，看前向缓冲区的开头字符串
2. 在搜索窗口中寻找最长匹配串
3. 如果找到匹配，就输出 `(off, len, c)`
4. 如果找不到匹配，就输出 `(0, 0, c)`
5. 窗口向右滑动 `len + 1` 个字符
6. 重复以上过程，直到输入结束

每一步的意图

第一步，看当前要编码的内容：

确定现在要压缩哪一段

第二步，找最长匹配：

匹配越长，能省掉的字符越多

第三步，输出指针和长度：

用“在哪里 + 复制多长”代替重复字符串

第四步，补上字符 `c`：

保证即使匹配结束，后面那个新字符也能被编码进去

第五步，滑动窗口：

已经编码过的数据变成新的历史数据，

后面的数据继续进入前向缓冲区

8. LZ77 计算例子

设窗口大小为 10。

已经编码过的 10 个字符是：

```
abcdbbccaa
```

即将编码的字符是：

```
abaeaaabaee
```

初始状态可以看成：

搜索窗口：abcdbbccaa

前向缓冲：abaeaaabaee

第一步

前向缓冲区开头是：

```
abae...
```

在搜索窗口 abcdbbccaa 中找最长匹配。

可以找到：

```
ab
```

匹配长度：

```
len = 2
```

匹配后的下一个字符是：

```
c = a
```

所以输出：

```
(0, 2, a)
```

意思是：

从窗口中偏移 0 的位置复制 2 个字符 ab，
再补上 a。

这一步实际编码了：

```
aba
```

窗口向右滑动：

```
len + 1 = 3
```

第二步

现在前向缓冲区开头变成：

```
aaaabaee
```

开头字符 e 在窗口里找不到合适匹配。

所以输出：

```
(0, 0, e)
```

意思是：

没有匹配，直接输出 e。

窗口向右滑动：

```
0 + 1 = 1
```

第三步

现在前向缓冲区开头是：

```
aaabaee
```


在窗口中可以找到最长匹配：

```
aaabae
```

匹配长度：

```
len = 6
```

匹配后的下一个字符是：

```
c = e
```

所以输出：

```
(4, 6, e)
```

意思是：

从窗口中偏移 4 的位置复制 6 个字符 **aaabae**，
再补上 **e**。

这一步实际编码了：

```
aaabaee
```

窗口向右滑动：

```
6 + 1 = 7
```

LZ77 输出结果

所以这个例子可以编码为：

```
(0, 2, a)
```

```
(0, 0, e)
```

```
(4, 6, e)
```

原来要写很多字符，现在变成几个三元组。

这就是 LZ77 的压缩来源。

9. LZ77 解码怎么做

LZ77 解码比编码简单。

看到一个三元组：

```
(off, len, c)
```

就做：

1. 在已经解码的数据窗口中找到 `off` 对应的位置
2. 从那里复制 `len` 个字符
3. 再输出字符 `c`
4. 更新窗口

如果看到：

```
(0, 0, c)
```

就说明：

没有匹配串，直接输出 `c`

解码为什么简单

因为编码端已经把“在哪里复制、复制多长、后面补什么”都写进三元组了。

解码器只需要照着执行。

10. LZ77 的问题

LZ77 有两个常见冗余问题。

问题一：空指针冗余

如果没有匹配，LZ77 仍然输出：

```
(0, 0, c)
```

但其中：

0, 0

其实没提供太多有效信息。

这就是空指针冗余。

问题二：额外字符冗余

LZ77 的输出格式固定带一个后继字符 `c`。

有时这个 `c` 可能本来可以作为下一次更长匹配的一部分。

但因为当前三元组必须带上它，就可能影响后续匹配效果。

所以 LZ77 虽然思想清楚，但输出格式还可以改进。

LZSS 就是对 LZ77 的改进。

11. LZSS 是什么

LZSS 也是第一类词典编码。

它可以看成：

改进版 LZ77

它的核心改进是：

不是每次都输出三元组。

只有当匹配真的值得压缩时，才输出指针。

否则直接输出原字符。

12. LZSS 的输出方式

LZSS 通常有两种输出：

真实字符

指针 (Pointer, Length)

但解码器必须知道当前输出到底是哪一种。

所以 LZSS 还需要一个标志位：

标志位 + 数据

例如可以约定：

0 + 字符 表示直接输出字符
1 + 指针长度 表示复制历史窗口中的字符串

具体 0/1 怎么规定，不同实现可以不同，关键是接收端要知道规则。

13. LZSS 怎么计算

LZSS 编码步骤如下：

1. 把编码位置放到输入数据流开头
2. 在搜索窗口中查找前向缓冲区开头的最长匹配串
3. 得到匹配指针 **Pointer** 和匹配长度 **Length**
4. 判断 **Length** 是否大于等于最小匹配长度 **MIN_LENGTH**
5. 如果 **Length** $\geq$ **MIN_LENGTH**，输出指针
6. 如果 **Length** $<$ **MIN_LENGTH**，输出真实字符
7. 移动编码位置
8. 重复，直到输入结束

每一步的意图

第一步，确定编码位置：

知道当前处理到哪里

第二步，找最长匹配：

尽量找到可复用的历史字符串

第三步，得到 **Pointer** 和 **Length**：

准备判断“用指针是否划算”

第四步，引入 **MIN_LENGTH**：

避免为了很短的匹配反而输出更长的指针

例如只匹配到 1 个字符：

直接写这个字符

可能比输出一个指针更省。

第五步，匹配足够长才输出指针：

真正利用重复字符串节省空间

第六步，匹配太短就输出真实字符：

避免空指针和短匹配带来的冗余

这就是 LZSS 相比 LZ77 压缩效果更好的原因。

14. LZSS 计算例子

输入数据流：

A A B B C B B A A B C

设：

MIN_LENGTH = 2

也就是说：

匹配长度至少为 2，才值得输出指针

编码过程可以写成：

步骤	位置	匹配串	输出
1	1	无	A
2	2	无	A
3	3	无	B

步骤	位置	匹配串	输出
4	4	BB	(3, 2)
5	6	无	C
6	7	AAB	(7, 3)
7	11	无	C

最终输出大概是：

```
A
A
B
(3, 2)
C
(7, 3)
C
```

实际存储时还会加标志位，用来区分：

```
这是字符
还是指针
```

例如可以理解成：

```
字符 A：直接写
指针 (3,2)：回到历史窗口中某处，复制 2 个字符
```

这个例子的关键点

前三个字符：

```
A A B
```

前面没有足够长的匹配，所以直接输出。

第 4 步开始，发现：

```
BB
```

可以由历史数据得到，而且长度为 2，满足 `MIN_LENGTH = 2`。

所以输出指针：

(3, 2)

后面发现：

AAB

也能匹配，长度为 3。

所以输出：

(7, 3)

这样就避免重复写 BB 和 AAB。

15. LZ77 和 LZSS 的区别

对比点	LZ77	LZSS
输出格式	固定输出 (off, len, c)	输出字符或指针
无匹配时	仍输出 (0, 0, c)	直接输出字符
短匹配时	也可能输出三元组	短匹配直接输出字符
是否需要标志位	通常不需要额外区分	需要标志位区分字符/指针
压缩效果	较基础	通常比 LZ77 更好
核心改进	滑动窗口找重复	判断“输出指针是否划算”

最关键的一句：

LZ77：不管划不划算，都按三元组输出。

LZSS：只有划算时才输出指针，否则直接输出字符。

16. 词典编码和熵编码的关系

词典编码和熵编码不是一回事。

词典编码解决的是：

重复字符串怎么表示得更短

熵编码解决的是：

不同符号怎么根据概率分配更短的二进制码

所以它们可以联合使用。

比如：

先用 LZSS 找重复串，输出字符和指针
再用 Huffman 编码进一步压缩这些输出符号

很多实际压缩工具就是组合思路：

LZ 类算法 + Huffman/其他熵编码

17. 每一步的总意图

步骤	做什么	意图
建立历史窗口	保存已经编码过的数据	作为可查找的词典
查看前向缓冲区	确定当前要编码的字符串	知道要匹配什么
寻找最长匹配	在历史窗口找重复串	重复越长，压缩越明显
输出指针	用位置和长度代替重复串	减少重复字符存储
输出真实字符	没有有效匹配时直接写字符	避免指针冗余
设置 MIN_LENGTH	判断指针是否值得输出	避免短匹配越压越大
滑动窗口	更新历史数据和待编码数据	继续处理后续输入

考试背诵版

可以这样背：

词典编码是一类利用数据中重复字符串进行压缩的无损编码方法，其核心思想是用指针或索引代替重复出现的字符串。词典编码分为两类：第一类词典法在已经编码过的数据中查找当前字符串是否出现过，若出现则输出指向历史位置的指针，代表算法有 LZ77 和 LZSS；第二类词

典法在编码过程中动态建立短语词典，遇到已有短语时输出该短语在词典中的索引号，代表算法有 LZ78 和 LZW。LZ77 使用滑动窗口，在历史窗口中寻找前向缓冲区开头的最长匹配串，并输出三元组 (off, len, c)，表示从历史窗口某位置复制 len 个字符并补上后继字符 c。LZSS 是对 LZ77 的改进，它不再固定输出三元组，而是根据匹配长度判断是否值得输出指针；当匹配长度大于等于 MIN_LENGTH 时输出指针，否则直接输出真实字符，并用标志位区分字符和指针。因此 LZSS 减少了 LZ77 中的空指针冗余和短匹配冗余，通常能获得更好的压缩效果。

最关键的一句：

第一类词典法：前面出现过，就用指针指回去。

第二类词典法：词典里已经有，就输出词典索引。

LZ77 固定输出三元组，**LZSS** 判断划算才输出指针。